Dhanvantari AI Doctor

System Architecture, Router Mappings & Consultation Execution Flow

Project Name:	Dhanvantari (AI Doctor Triage & Chatbot)	**Execution Context:**	FastAPI, LangGraph State Engine, SQLite
Compliance Standard:	DPDP Act 2023 (India) & Telemedicine Guidelines	**Primary Model:**	MedGemma 1.5 & MedASR (Speech Engine)
Document Type:	Technical Reference / Router Map	**Last Generated:**	August 2026

1. System Design & Triage Objectives

The Dhanvantari AI Doctor is built as a stateful, safety-guaranteed digital triage system designed to consult with patients, analyze symptoms, and connect them with Registered Medical Practitioners (RMPs) when escalated. Two key constraints dictate the layout and operational mechanics of the application:

- **India's Telemedicine Practice Guidelines (2020) Compliance:** Autonomous AI is legally barred from issuing medical prescriptions. Hence, the system is designed strictly for triage, information distribution, and *drafting* prescriptions. Every diagnostic outcome and medication recommendation above self-care requires a human RMP to verify, digitally sign, and issue the final authorization.
- **DPDP Act 2023 Compliance:** Personal health data is treated with maximum protection. Patient consent scope and authorization date are verified at the gateway. No conversation session can be initialized or message processed without active consent logged in the backend database.

Compliance Note: *If deploying in the US, compliance parameters are mapped to HIPAA requirements, enforcing active Business Associate Agreements (BAAs), database encryption at rest and in transit, and secure access log trails for Protected Health Information (PHI). The state routing logic remains identical.*

2. End-to-End Consultation Life-Cycle

A standard patient consultation is executed via the following chronological sequence:

Step 1: Consent Registration: The patient registers their explicit consent scope (e.g. details, duration) via the auth route, logging patient records in SQLite.

Step 2: Client Engagement: The client initiates a chat turn by sending patient and conversation IDs with user input. The router performs a consent verification check against the database.

Step 3: State Reassembly: The chat router fetches the historical transcript from the DB to reconstruct the state context, passing it to the LangGraph machine.

Step 4: Safety & RAG Flow: The state machine executes in series: intake configurations, regex symptom parsing, emergency check (detouring immediately if triggered), RAG database matching, and LLM drafting.

Step 5: Triage Urgency Evaluation: The triage node determines the severity level (self_care, see_doctor_soon, urgent, emergency). Level 'self_care' completes the flow, while other levels queue an escalation item.

Step 6: RMP Sign-off Integration: For escalated sessions, the system generates an AI draft prescription template. An RMP logs into the admin portal, reviews the full chat history, signs the prescription, and resolves the queue item.

3. REST API Router Mapping

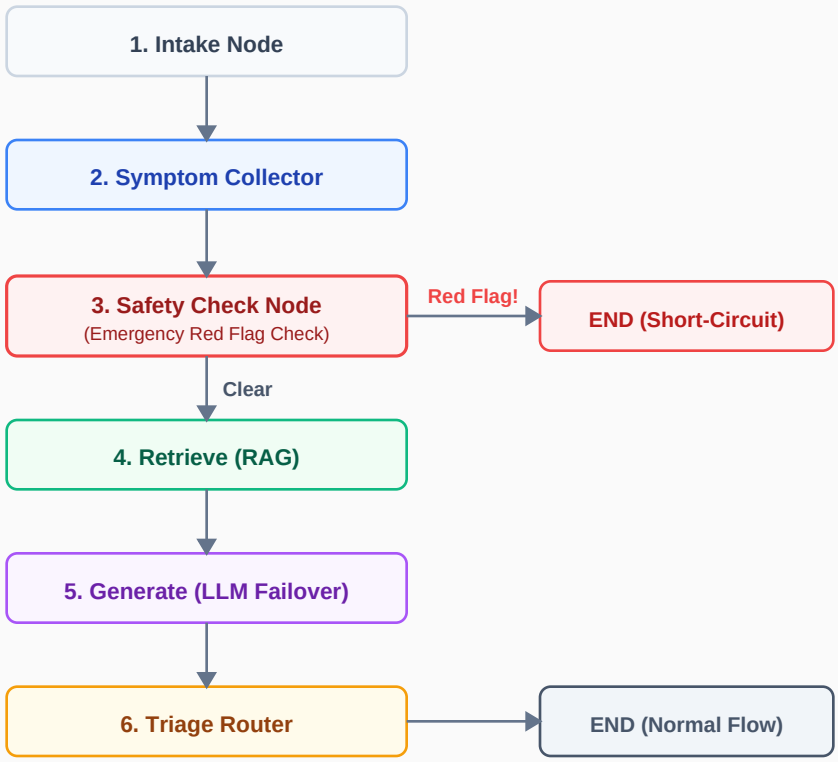
The project implements three core APIRouters (Auth, Chat, Admin) under the **/api/v1** prefix. The table below details all active endpoints, their handlers, data validations, and database impacts:

Endpoint & Method	Method	Handler / Source File	Description & Database Impact
/auth/consent	POST	capture_consent app/routers/auth.py	Registers a new patient and logs DPDP consent metadata (consent_scope, timestamp). If patient exists, updates their records.
/chat	POST	process_chat app/routers/chat.py	Validates consent (blocks 403 if missing). Reconstructs conversation history, invokes State Machine, logs messages, updates triage flags.
/admin/queue	GET	get_review_queue app/routers/admin.py	Retrieves escalated consultations waiting in the ReviewItem queue, listing patient email, current triage level, and timestamp.
/admin/conversation/{conversation_id}	GET	get_conversation_detail app/routers/admin.py	Returns detailed chat transcript, triage level, patient data, and DPDP consent logs for a specific escalated consultation.
/admin/prescription/submit	POST	submit_prescription app/routers/admin.py	Accepts RMP registration, signature, and final prescription. Marks ReviewItem as 'prescribed', updates Conversation to 'resolved'.

4. LangGraph Triage State Machine Flow

At the core of the consultation processing is a state machine built using a lightweight StateGraph compiler (matching LangGraph's state machine API). The flow executes sequentially, except when red flags trigger conditional routing to abort processing and output immediate emergency disclaimers:

LangGraph Triage State Machine execution pathways



Node Name	Function Location	Key Inputs & Outputs	Behavior & Logic Summary
intake	intake.run nodes/intake.py	In: State dict Out: Init defaults	Ensures necessary key arrays (symptoms, retrieved_context) and status defaults are configured.
collect_symptoms	symptom_collector.run nodes/symptom_collector.py	In: turns list Out: symptoms list	Uses regex patterns to parse symptom descriptions, onset times (e.g. 3 days), and severity scale (1-10) from the latest user message.
safety_check	safety_check.run nodes/safety_check.py	In: last message Out: red_flag flag	Matches input text to high-risk keywords. If positive, sets red_flag_detected=True, drafts emergency directions, and triggers conditional END.
retrieve	retrieve.run nodes/retrieve.py	In: query text Out: retrieved_context	Fetches medical corpus standard actions (ICD-10 mapping, triage guidelines) to ground downstream generation.
generate	generate.run nodes/generate.py	In: context & turns Out: draft_response	Invokes LLMRouter (MedGemma primary, Gemini fallback). Parses structured JSON containing display and speech responses (Hinglish/English).
triage	triage_router.run nodes/triage_router.py	In: symptoms, context Out: triage_level, RMP queue	Classifies severity (urgent, see_doctor_soon, self_care). If higher than self_care, escalates the case by inserting it into ReviewItem.

5. Safety Guardrails & Short-Circuit Mechanics

A critical feature of the system is the deterministic emergency guardrail inside **safety_check**. Instead of relying on unstable LLM semantic analysis which is prone to hallucination or prompt injection, the safety check runs **prior to any LLM execution** as a hard-coded pattern match.

If the system detects critical keywords corresponding to venomous bites, cardiac events, stroke, suicidal risk, breathing distress, or high pediatric fevers, the following happens immediately:

1. The conversation status changes to **emergency** and **red_flag_detected** is set to True.
2. The system loads pre-written first-aid instructions specific to the emergency type (e.g. immobilizing the limb for snake bites, calling 108/112 in India or 911 in the US).
3. The LangGraph routing function selects the **emergency** conditional branch, routing directly to **END**. This completely bypasses the RAG retrieval and LLM generation phases, preventing latency delays and ensuring unaltered, compliant first-aid instructions are returned to the user.

6. RMP Review Queue & Escalation Loop

When the triage node classifies a session at a severity level higher than self-care (urgent, see_doctor_soon, emergency), it initializes an escalation pipeline:

- **Draft Prescription Template:** The system creates a ReviewItem record containing an AI-generated draft prescription template. This template standardizes critical items (Triage Level, Symptoms, Rx slot, Clinical Advice, and RMP Registration details) but leaves medical prescriptions blank, marked for RMP authorization.
- **Doctor Review Queue:** The consultation session status is updated to *escalated*. Doctors use the admin queue route to pull pending items. RMPs can fetch full transcripts and patient consent scopes to understand context.

- **Sign-off and Resolution:** The doctor reviews the cases, modifies or adds prescription medication, input their name and RMP registration number, and submits the payload. Once submitted, the ReviewItem is flagged as *prescribed*, the parent conversation is marked as *resolved*, and the consultation loop is completed.